Content Management System (CMS) for Student Data Management

ABSTRACT

This project is a user-friendly Content Management System (CMS) developed in C++ to simplify student data management. The system allows teachers to easily add, update, and view student records such as grades and attendance. It also enables students to view their own academic information using their student ID. The program uses file handling to store and retrieve data, ensuring that records are saved permanently. A password-protected teacher menu ensures secure access. The CMS improves efficiency, accuracy, and usability for managing student information.

INTRODUCTION

Managing student records manually can be difficult, time-consuming, and prone to errors. This project provides a simple console-based CMS using C++ that automates the process of managing student information. It allows teachers to maintain records efficiently and students to track their academic progress.

The system uses structures to store student data, arrays to hold multiple student records, file handling for permanent storage, a menu-driven interface using loops, and password authentication for teachers.

OBJECTIVES

- To develop a CMS for managing student records.
- To allow teachers to add and update student information.
- To enable students to view their grades and attendance.
- To use file handling for data storage.
- To design a simple, user-friendly menu interface.

- To ensure secure access for teachers using a password.

SYSTEM FEATURES

- Teacher authentication with password.
- Add new student records.
- Update existing student grades and attendance.
- Display all student data.
- Students can view their own record using ID.
- Data stored in a file (`students.txt`).
- Continuous interaction using do-while loops.

DATA STRUCTURE USED

A structure is used to store student information including Student ID, Student Name, Grade, and Attendance. An array of structures is used to store multiple student records in memory.

FILE HANDLING

The program uses file handling to load data from `students.txt` when the program starts and to save updated data back to the file when the program exits or updates occur. If the file does not exist, default student records are loaded.

PROGRAM WORKING

When the program starts, it tries to read student data from the file. If the file is not found, default student records are loaded.

The Main Menu displays three options: Teacher Menu, Student Menu, and Exit.

The Teacher Menu is password protected. After successful authentication, the teacher can view all student records, add new students, or update existing student information.

The Student Menu allows students to enter their ID and view their name, grade, and attendance.

All changes made during the program execution are saved to the file before exiting.

FUNCTION DESCRIPTIONS

loadDataFromFile()
Loads student data from the file or uses default values.

saveDataToFile()
Saves all student records to the file.

showAllStudents()
Displays grades and attendance of all students.

addStudent()
Adds a new student to the array.

updateStudentInfo()
Updates grade and attendance of a student using their ID.

authenticateTeacher()
Checks teacher password for secure access.

showStudentInfoById()
Displays student record based on entered ID.

ALGORITHM

Step 1: Start the program.

Step 2: Call `loadDataFromFile()`.
If file exists, load data. Otherwise, load default data.

Step 3: Display Main Menu:

1. Teacher Menu
2. Student Menu
3. Exit

Step 4: If Teacher Menu is selected:

- Ask for password using `authenticateTeacher()`.
- If correct, show Teacher options:
    - Show all students
    - Add new student
    - Update student info
    - Exit Teacher Menu
- Repeat until teacher exits.

Step 5: If Student Menu is selected:

- Ask for a student ID.
- Display student grade and attendance.
- Return to Main Menu.

Step 6: If Exit is selected:

- Call `saveDataToFile()`.
- End program.

TOOLS AND CONCEPTS USED

- C++ Programming Language
- Structures
- Arrays
- File Handling (`ifstream`, `ofstream`)
- Functions

- Loops (do-while)
- Conditional statements
- Password authentication

CONCLUSION

The CMS for student data management provides essential features for both teachers and students. Teachers can manage student records efficiently, while students can easily track their academic performance. The use of file handling ensures permanent data storage, and password protection maintains security. The menu-driven design makes the system simple, interactive, and user-friendly.

FUTURE IMPROVEMENTS

- Add delete student feature.
- Add multiple subjects and automatic result calculation.
- Convert the program into a graphical user interface (GUI).
- Use a database instead of a file for data storage.
- Improve input validation and error handling.